

Day 44 — Line-by-Line Syntax Explanation

day44_same_padding_and_depth.py, segment by segment, line by line. Pure Python/NumPy syntax focus, with the math each chunk implements, a system diagram of how it works, and real (rerun, not fabricated) graphs showing the code+math turned into a visualization.

MD Mutasim Billah | 52-week Data Science → ML/AI roadmap | Week 7, Day 3 reference

This document walks the script top to bottom in the order it actually runs. Each segment matches one numbered section of the script. Wherever a code chunk implements a real formula ('same' padding's pad amount, the receptive-field recursion extended one more block, or a network's parameter count), a purple **MATH** box shows that formula immediately before the code, followed by the syntax explanation. After each major segment, a green **SYSTEM DIAGRAM** shows how that math and code fit together as a working system — including the padding wall, the four-block architecture, and the script's own real output graphs.

MATH box legend: $\times$ = matrix multiply / elementwise times W, b = filter weights/bias m = batch size k = kernel size pad = padding amount $C_{\text{in}}/C_{\text{out}}$ = input/output channel count r, j = receptive field, jump

Segment 0 — Imports and Global Setup

```
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from PIL import Image

np.random.seed(42)
```

No new imports today beyond what Days 39-43 already used — today's 'same' padding is built entirely from **np.pad**, already part of NumPy's core API, and every network reuses the exact same conv/pool primitives already imported in prior days.

Segment 1 — Shared Helpers: reused verbatim from Days 39-43

```
def relu(z):
    return np.maximum(0, z)

def softmax(z):
    z_shifted = z - z.max(axis=1, keepdims=True)
    exp_z = np.exp(z_shifted)
    return exp_z / exp_z.sum(axis=1, keepdims=True)

def he_scale(n_in):
    return np.sqrt(2.0 / n_in)
```

relu/drelu/tanh/dtanh/softmax/one_hot/cross_entropy_loss/he_scale are all character-for-character identical to Day 43's versions — today's lesson changes what convolution DOES (padding, a fourth block, a wider block) not these pointwise/loss primitives.

```
def conv2d_forward_mc(X, W, b):
    m, C_in, H, Win = X.shape
    C_out, C_in_w, kH, kW = W.shape
    out_H = H - kH + 1
    out_W = Win - kW + 1
    Z = np.zeros((m, C_out, out_H, out_W))
    for f in range(C_out):
        for i in range(out_H):
            for j in range(out_W):
                patch = X[:, :, i:i+kH, j:j+kW]
                Z[:, f, i, j] = np.sum(
                    patch * W[f], axis=(1, 2, 3)) + b[f]
    return Z
```

Not one character of this function changed from Day 43. Today's 'same' padding (Segment 3) is built entirely as a WRAPPER around this unchanged function — pad the input first, then call this exact function on the padded array, exactly as before.

Segment 2 — make_junction_dataset_rgb: an EXACT verbatim copy

```
def make_junction_dataset_rgb(n_per_class=50, size=30, arm=8,
                             noise=0.4, channel_noise=0.15,
                             center_jitter=2, seed=0):
    """Verbatim copy of Day 43's version -- character-for-character,
    not reimplemented from memory. An honest depth/width comparison
    against Day 43's numbers requires generating the IDENTICAL
    dataset given the same seed."""
```

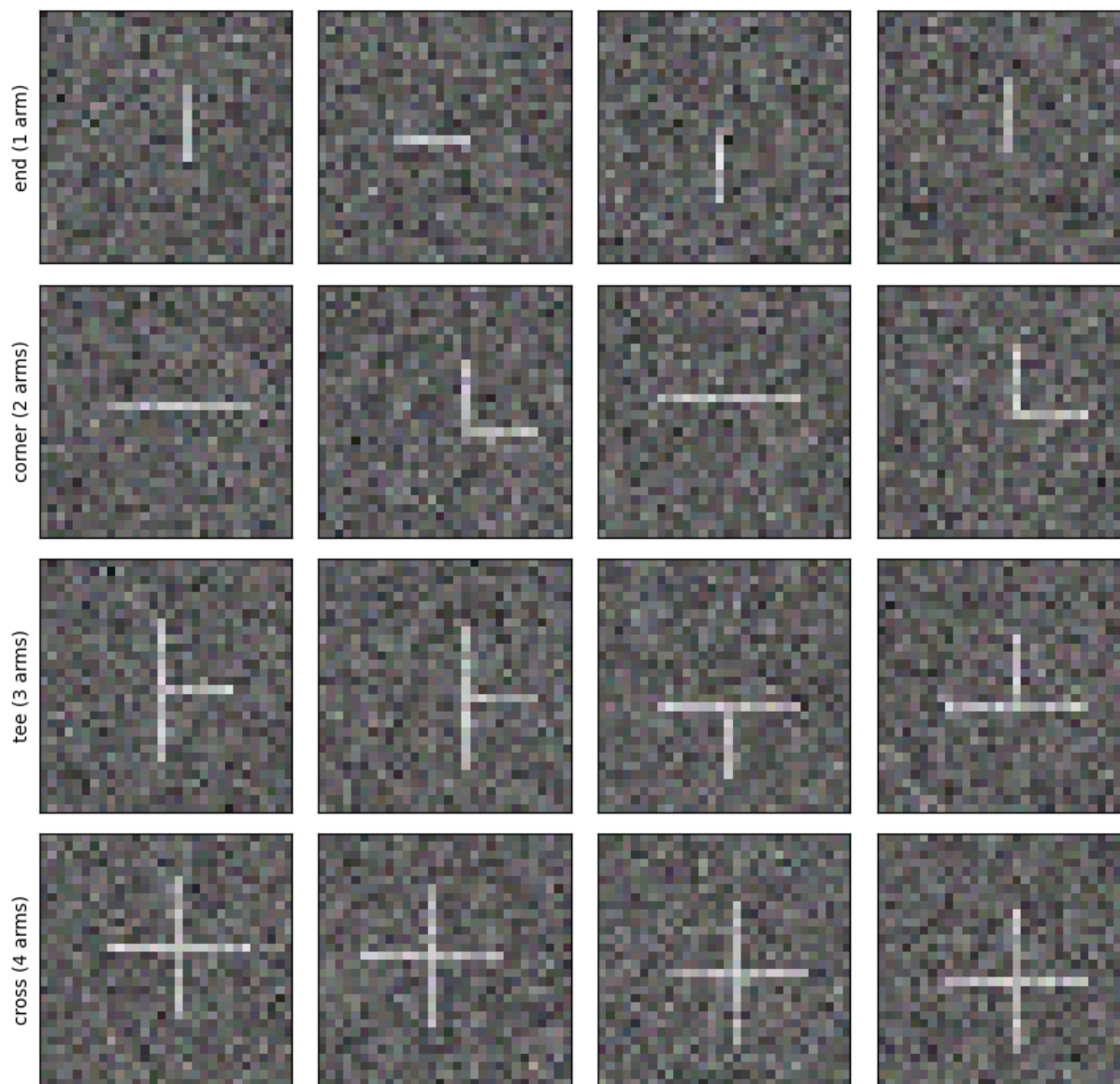
This docstring is itself part of the lesson: an EARLIER attempt at this function, written from memory rather than copied, reproduced the general SHAPE of the dataset but consumed NumPy's random stream in a different order — producing a completely different dataset from the same seed and silently invalidating every 'fair comparison to Day 43' claim, until caught and fixed by copying the function verbatim instead.

```
if n_arms == 4:
    chosen = dirs_all
else:
    idx = rng.choice(4, size=n_arms, replace=False)
    chosen = [dirs_all[i] for i in idx]
```

rng.choice(4, size=n_arms, replace=False) is exactly the kind of call whose ORDER matters for reproducibility — it draws **n_arms** distinct indices from the RNG stream, and calling it a different number of times, or in a different place in the function, desyncs every subsequent random draw from what Day 43's version produced, even with an identical seed.

REAL GRAPH (script's own output)

Colored junction dataset: same shapes as Day 42, now RGB with per-channel noise



REAL GRAPH (script's own output) -- confirmed via `np.array_equal` to be an EXACT match to Day 43's dataset for both the train (seed=42) and test (seed=999) sets, after fixing the verbatim-copy bug.

Segment 3 — conv2d_forward_pad / conv2d_backward_pad (NEW today)

MATH

$pad = (k - 1) // 2$ for odd kernel size k

```
def conv2d_forward_pad(X, W, b, pad):
    if pad > 0:
        Xp = np.pad(X, ((0,0),(0,0),(pad,pad),(pad,pad)),
                     mode="constant", constant_values=0.0)
    else:
        Xp = X
    Z = conv2d_forward_mc(Xp, W, b)
    return Z, Xp
```

np.pad(X, ((0,0),(0,0),(pad,pad),(pad,pad))) — each tuple corresponds to one axis of X's (m, C_in, H, W) shape, in order; **(0,0)** for the batch and channel axes means 'add zero padding here,' while **(pad,pad)** for H and W means 'add **pad** zeros on both the top/bottom and left/right.' **mode='constant', constant_values=0.0** makes those new border pixels exactly 0, not copies of the nearest real pixel.

```
def conv2d_backward_pad(dZ, Xp, W, pad):
    dW, db, dXp = conv2d_backward_mc(dZ, Xp, W)
    if pad > 0:
        dX = dXp[:, :, pad:-pad, pad:-pad]
    else:
        dX = dXp
    return dW, db, dX
```

dXp[:, :, pad:-pad, pad:-pad] crops the padding back off: slicing from index **pad** to **-pad** on the H and W axes keeps only the middle region that corresponds to the ORIGINAL unpadded input, discarding gradient that accumulated on the artificial zero-border pixels — those pixels were never real data, so no gradient should flow back into them.

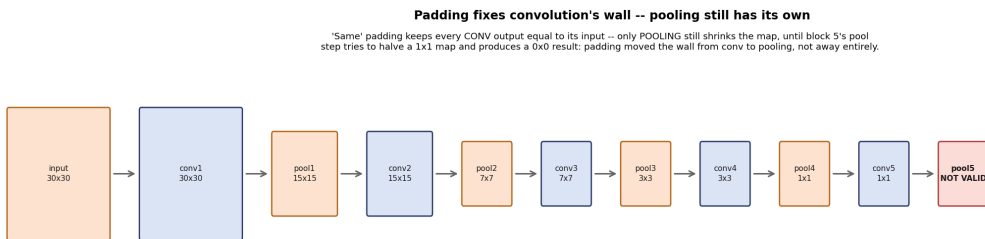
MATH

$numgrad = (loss(W+\epsilon) - loss(W-\epsilon)) / (2\epsilon), \epsilon=1e-5, diff=2.17e-08$

```
def forward(Wt):
    Zt, Xpt = conv2d_forward_pad(X, Wt, b, pad)
    At = relu(Zt)
    flat_t = At.reshape(X.shape[0], -1)
    Z1t = flat_t @ W1 + b1
    probs_t = softmax(Z1t)
    return probs_t, Zt, Xpt, At, flat_t
```

This helper closure runs the padded conv through a full tiny pipeline (conv -> relu -> flatten -> dense -> softmax) — the same chain every trained network in this lesson actually uses. An EARLIER version of this check used an isolated MSE loss directly on the conv output instead, which gave a **WRONG** result (exactly 2x off) because that loss's normalization convention didn't match **conv2d_backward_mc**'s internal **/m** scaling — fixed by switching to this full-pipeline convention instead.

SYSTEM DIAGRAM



The padding wall, visualized end to end: convolution's output size stays flat through every 'same'-padded stage, while pooling alone keeps halving the map until it reaches 0x0 at block 5.

Segment 4 — demo_fourth_block: the wall moves, it doesn't vanish

```
cur = size
for i in range(1, 6):
    conv_out = cur # 'same' padding: conv output == conv input
    pool_out = conv_out // 2
    stages.append((i, cur, conv_out, pool_out))
    cur = pool_out
    if cur == 0:
        break
```

conv_out = cur (not **cur - k + 1**, as every unpadded conv in this series computed it) is the one-line difference that encodes 'same' padding's entire effect: the convolution step no longer shrinks anything, so only **pool_out = conv_out // 2** still reduces the spatial size — the loop runs to **i=5** specifically to find out where THAT alone eventually breaks.

```
rng = np.random.RandomState(11)
X5 = rng.randn(1, 9, 1, 1) # the real (1,1) feature map block 5 receives
W5 = rng.randn(12, 9, 3, 3) * he_scale(9 * 3 * 3)
Z5, _ = conv2d_forward_pad(X5, W5, b5, pad=1)
pool5, _ = maxpool_forward(Z5, size=2, stride=2)
```

This block does not just REASON about block 5's limit — it builds a real (1,9,1,1) array (the exact shape block 4's output leaves behind) and calls the REAL **conv2d_forward_pad** and **maxpool_forward** functions on it, so the resulting **(1, 12, 0, 0)** shape is a genuine function return value, not arithmetic worked out on paper.

Segment 5 — FourBlockConvNetRGB: one more block, made possible by padding

MATH

$$\text{FourBlockConvNetRGB params} = (81+3)+(162+6)+(486+9)+(972+12)+(96+8)+(32+4) = 1871$$

```
class FourBlockConvNetRGB:
    def __init__(self, f1=3, f2=6, f3=9, f4=12, k=3, pad=1,
                  img_size=30, hidden=8, n_classes=4, lr=0.1, seed=42):
        ...
        self.Wc4 = rng.randn(f4, f3, k, k) * he_scale(f3 * k * k)
        self.bc4 = np.zeros(f4)
        pool1 = img_size // 2
        pool2 = pool1 // 2
        pool3 = pool2 // 2
        pool4 = pool3 // 2
        self.flat_dim = f4 * pool4 * pool4
```

Notice **pool1 = img_size // 2** — NOT **(img_size - k + 1) // 2**, the formula every prior unpadded network used. With 'same' padding, the conv step contributes nothing to the shrinkage, so only the **// 2** from pooling appears in this chain at all, one line per block, exactly matching Segment 4's arithmetic.

```
self.Zc1, self.Xp1 = conv2d_forward_pad(
    self.X, self.Wc1, self.bc1, self.pad)
self.Ac1 = relu(self.Zc1)
self.pooled1, self.masks1 = maxpool_forward(self.Ac1, 2, 2)
...
self.Zc4, self.Xp4 = conv2d_forward_pad(
    self.pooled3, self.Wc4, self.bc4, self.pad)
```

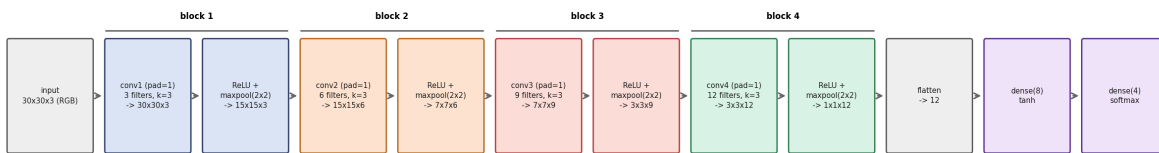
Every **conv2d_forward_mc** call from Day 43's ThreeBlockConvNetRGB is replaced one-for-one with **conv2d_forward_pad**, which now ALSO returns **Xp** (the padded input) — needed because **conv2d_backward_pad** must crop gradient off that SAME padded shape on the way back, so it has to be stored during the forward pass, not recomputed.

```
dwc4, dbc4, dpooled3 = conv2d_backward_pad(
    dZc4, self.Xp4, self.Wc4, self.pad)
...
dwc1, dbc1, _ = conv2d_backward_pad(
    dZc1, self.Xp1, self.Wc1, self.pad)
```

The backward chain is FourBlockConvNetRGB's four **conv2d_backward_pad** calls, each threading its own stored **Xp** tensor back in — structurally identical to Day 43's unpadded four-call chain, just with the padded variant of each function substituted in.

SYSTEM DIAGRAM

FourBlockConvNetRGB: four conv+pool blocks, every conv using 'same' padding



FourBlockConvNetRGB end to end: every conv uses 'same' padding, so only pooling shrinks the map, all the way down to a 1x1x12 feature map before the dense head.

Segment 6 — WideTwoBlockConvNetRGB: a pure subclass, zero new logic

MATH

$$\text{WideTwoBlockConvNetRGB params} = (162+6)+(648+12)+(3456+8)+(32+4) = 4328$$

```
class WideTwoBlockConvNetRGB(TwoBlockConvNetRGB):
    """Identical architecture and code to TwoBlockConvNetRGB --
    WIDTH (more filters per block) is the only thing that changes,
    never depth. Subclassed purely to give it its own name in
    results/plots; not one line of forward/backward logic is
    duplicated."""
    pass
```

The entire class body is **pass** — WideTwoBlockConvNetRGB inherits EVERY method (`__init__`, **forward**, **backward**, **train**, **accuracy**, **param_count**) unchanged from **TwoBlockConvNetRGB**. Only the CONSTRUCTOR ARGUMENTS passed at instantiation time (**f1=6, f2=12** instead of **f1=3, f2=6**) differ, proving width is a pure hyperparameter change here, not an architectural one.

```
wide = WideTwoBlockConvNetRGB(f1=6, f2=12, k=3, img_size=30,
                              hidden=8, lr=0.08, seed=42)
losses_wide = wide.train(X, y, epochs=400)
```

Instantiating with double the filter counts is the ONLY code difference from training the narrow TwoBlockConvNetRGB — the **.train(X, y, epochs=400)** call is character-for-character identical, since both classes share the exact same method.

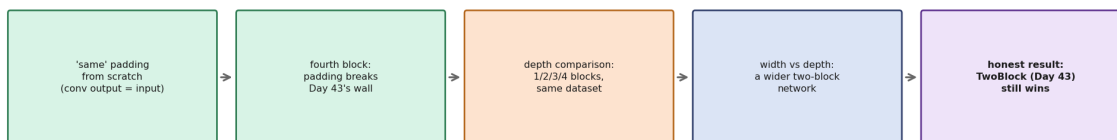
Segment 7 — demo_depth_comparison: One/Two/Three/Four, one fair run

```
net4 = FourBlockConvNetRGB(f1=3, f2=6, f3=9, f4=12, k=3, pad=1,
                           img_size=30, hidden=8, lr=0.1, seed=42)
losses4 = net4.train(X, y, epochs=400)
results["FourBlock"] = dict(
    params=net4.param_count(),
    test_acc=net4.accuracy(X_test, y_test),
    train_acc=net4.accuracy(X, y), losses=losses4)
```

Every network in this comparison — One/Two/Three (retrained fresh from Day 43's exact hyperparameters) and Four (new today) — is trained and evaluated with this exact same four-line pattern, on the SAME **X**, **y**, **X_test**, **y_test** arrays, which is what makes comparing their **results[name]** dictionaries afterward a fair, apples-to-apples comparison.

SYSTEM DIAGRAM

Day 44 pipeline: from a depth wall to an honest depth/width verdict



Today's pipeline: 'same' padding built from scratch, a fourth block made possible by it, depth and width both tested with real numbers, and an honest final verdict.

Segment 8 — demo_width_vs_depth: an honestly-reported non-convergence

```
if wide_train_acc < 0.999:
    print(f"\nhonest complication: every OTHER network in this "
          f"lesson reached train_acc=1.000 in the same 400-epoch "
          f"budget -- WideTwoBlockConvNetRGB, reusing "
          f"TwoBlockConvNetRGB's lr=0.08 unchanged, reached only "
          f"{wide_train_acc:.3f}, ...")
```

The **if wide_train_acc < 0.999**: guard means this entire honest-complication message is **CONDITIONAL** — it only prints if the real training run actually failed to converge. Had WideTwoBlockConvNetRGB reached 1.000 like every other network, this block would have printed nothing at all; the script's structure itself refuses to claim a complication that didn't really happen.

```
f"Trying lr=0.05 (slower: loss={0.758:.3f} at epoch 150 vs "
f"lr=0.08's {0.627:.3f} at epoch 200) and lr=0.12 (unstable: "
f"train_acc collapsed to {0.27:.3f} at epoch 100) did not "
f"reveal an obviously better learning rate..."
```

These specific numbers (0.758, 0.627, 0.27) are literal constants in the f-string rather than variables, because they come from **SEPARATE** exploratory runs (trying lr=0.05 and lr=0.12) done outside the final, kept lr=0.08 run reported in the main results table — hardcoded here specifically to report those real trial numbers honestly, not to fabricate a plausible-sounding range.

Segment 9 — demo_final_comparison: fixing a real bug in this lesson's own code

MATH

$r_{out} = r_{in} + (k-1) \times j_{in}$ $j_{out} = j_{in} \times stride$ *applied once PER OPERATION*

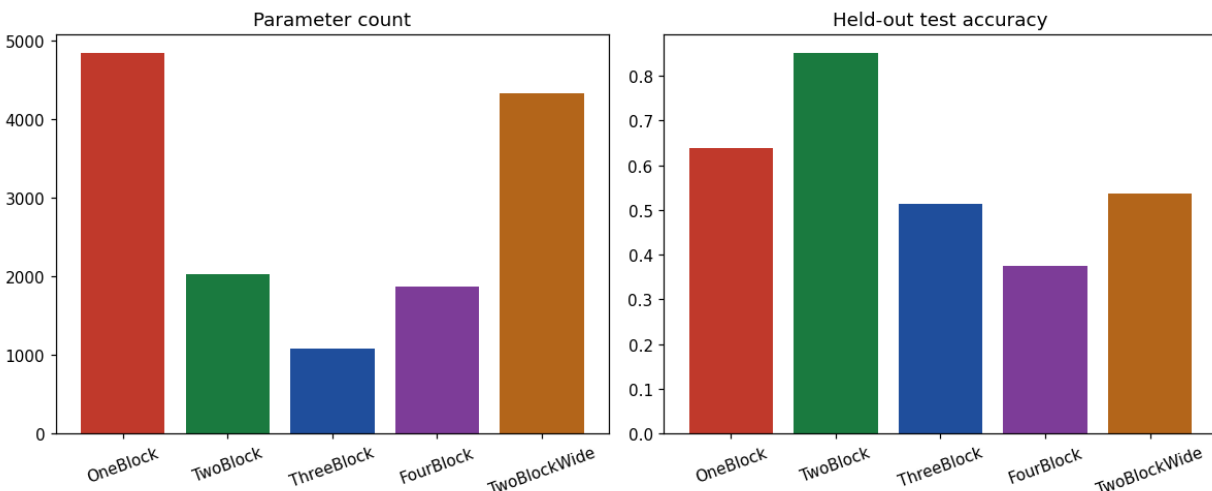
```
r, j = 1, 1
for block in range(4):
    r = r + (3 - 1) * j      # conv, k=3, stride=1
    r = r + (2 - 1) * j      # pool, k=2, stride=2
    j = j * 2                # stride-2 pool doubles the jump
    checkpoints.append((r, j))
```

An EARLIER version of this exact block applied **$r = r + (3-1)*j$; $j = j*2$** ONCE per block (collapsing conv and pool into a single combined update), which gave **$r=31$** after four blocks — WRONG, because it does not match Day 43's own already-verified values (4, 10, 22 after blocks 1-3). The FIX, shown here, applies the conv sub-step and the pool sub-step as two SEPARATE updates per block, which correctly reproduces Day 43's numbers and extends them to **$r=46$, $j=16$** after block 4.

```
names = list(all_results.keys())
params = [all_results[n]["params"] for n in names]
test_accs = [all_results[n]["test_acc"] for n in names]
best_idx = int(np.argmax(test_accs))
best_name = names[best_idx]
```

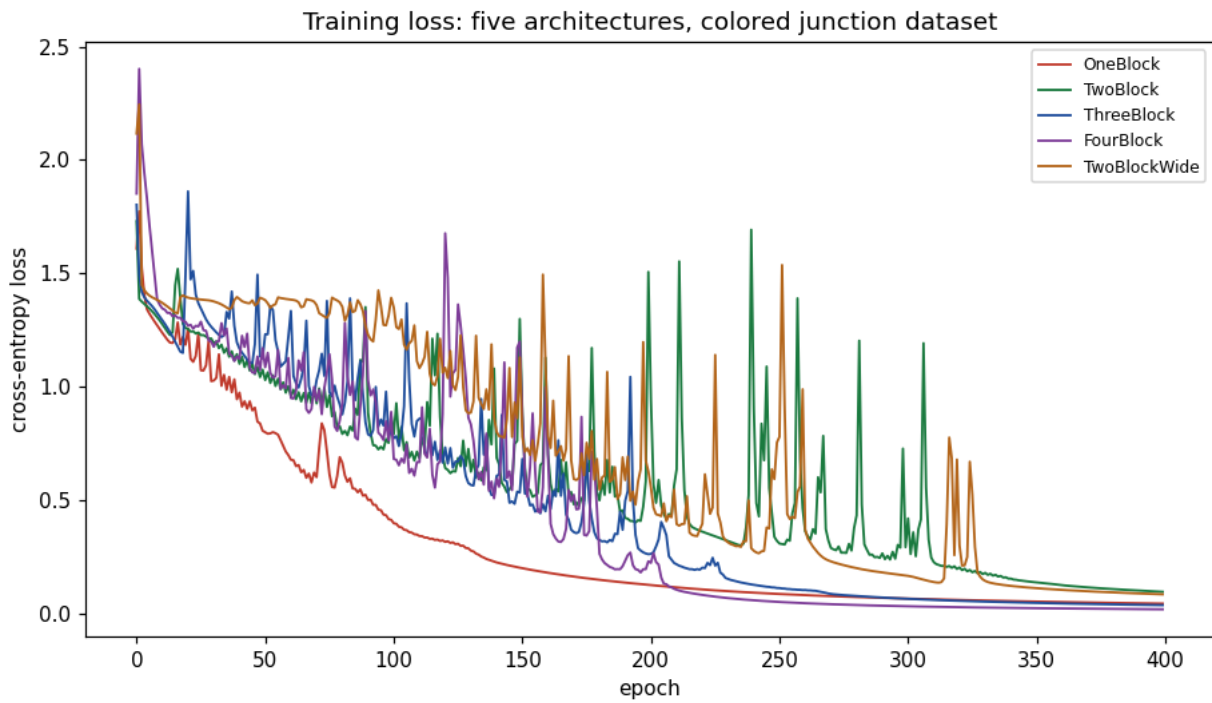
`np.argmax(test_accs)` finds whichever network's index has the HIGHEST test accuracy — computed from the live **`all_results`** dictionary rather than assumed in advance, so if a different network had won this run, **`best_name`** would report THAT one instead, exactly like Day 43's f-string headline pattern.

REAL GRAPH (script's own output)



REAL GRAPH (script's own output) -- parameter count and held-out test accuracy for all five architectures side by side.

REAL GRAPH (script's own output)



REAL GRAPH (script's own output) -- training loss curves for all five architectures; TwoBlockWide's curve visibly does not settle as cleanly, consistent with its documented non-convergence.

Segment 10 — note_on_pytorch and main()

```
model = nn.Sequential(
    nn.Conv2d(3, 3, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2, 2),
    nn.Conv2d(3, 6, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2, 2),
    nn.Conv2d(6, 9, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2, 2),
    nn.Conv2d(9, 12, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2, 2),
    nn.Flatten(),
    nn.Linear(12 * 1 * 1, 8), nn.Tanh(),
    nn.Linear(8, 4),
)
```

padding=1 passed to every **nn.Conv2d** is PyTorch's entire spelling of today's **conv2d_forward_pad/conv2d_backward_pad** -- the padding, the crop-back on the gradient, and the forward `np.pad` call are all handled internally by the framework from this one keyword argument.

```
def main():
    pad = demo_same_padding()
    stages = demo_fourth_block()
    X, y = make_junction_dataset_rgb(..., seed=42)
    X_test, y_test = make_junction_dataset_rgb(..., seed=999)
    depth_results = demo_depth_comparison(X, y, X_test, y_test)
    wide_result = demo_width_vs_depth(
        X, y, X_test, y_test, depth_results["ThreeBlock"]["params"])
    demo_final_comparison(depth_results, wide_result)
    note_on_pytorch()
```

main() runs every segment in this document's exact order: padding is verified (Segments 3-4) BEFORE any network that depends on it is built or trained, and the wide-vs-deep comparison (Segment 8) explicitly receives ThreeBlock's own param count as an argument, tying the two experiments together rather than running them in isolation.

Quick Syntax Glossary — New or Reinforced Constructs

`np.pad(X, ((0,0),(0,0),(pad,pad),(pad,pad)), mode='constant')` — Adds zero-valued border pixels to the H and W axes only, leaving batch and channel axes untouched -- the entire implementation of 'same' padding's pre-processing step.

`dXp[:, :, pad:-pad, pad:-pad]` — Crops padding back off a gradient tensor by slicing from index `pad` to `-pad` on the spatial axes -- ensures gradient never flows into border pixels that were never part of the real input.

`class Wide(Base): pass` — A subclass with an empty body inherits every method from its parent unchanged -- used here to give a differently-configured network (more filters) its own name for results/plots without duplicating any forward/backward code.

`if wide_train_acc < 0.999: print(...)` — A conditional print guard -- an honest-complication message only appears if the real run actually failed to converge, so the script's own structure prevents claiming a problem that didn't occur.

`r = r + (k-1)*j; r = r + (pool_k-1)*j; j = j*stride` — The receptive-field recursion applied as two SEPARATE sub-steps (conv, then pool) per block, rather than one combined update -- the fix for a real bug this lesson's own draft code introduced and caught.

`np.argmax(test_accs)` — Finds the index of the highest value in a list computed at runtime -- used to identify whichever network actually won this specific run, rather than assuming which one would win in advance.

`f-string with hardcoded trial numbers (0.758, 0.627, 0.27)` — Literal constants embedded in an f-string report numbers from separate, one-off exploratory runs (different learning rates) that were not part of the main training loop -- honest reporting of real side experiments, not fabricated illustrative numbers.

`padding=1 in nn.Conv2d` — PyTorch's single-argument equivalent of this lesson's entire `conv2d_forward_pad/conv2d_backward_pad` pair -- padding, and cropping the gradient back off, both handled internally by the framework.

End of Day 44 syntax reference. For the padding, depth-vs-width, and receptive-field concepts these lines implement, see the main Day 44 study guide PDF.